Quiz #7 (Unit 7 - Extended): MIPS Instruction Encoding & Datapath Execution

Total Marks: 315.00

R-Type Bitfield Encoding, Variable Shifts, Flag Evaluation & Register Arithmetic

Name: **Student ID:** **Score:** / 315.00

> **Instructions:** Answer all 26 questions across the three primary parts. Refer to the reference table for MIPS register numbers and funct codes. Strictly adhere to the question formats: *[Fill in the Blank]* for machine code bit strings and numbers, *[Multiple Choice]* for single-option concept questions, and *[Select All That Apply]* for multi-option concept questions. Calculators are not permitted.

Field	Opcode (bits 31-26)	rs (bits 25-21)	rt (bits 20-16)	rd (bits 15-11)	shamt (bits 10-6)	funct (bits 5-0)
Bit Width	6 bits	5 bits	5 bits	5 bits	5 bits	6 bits
R-Type Role	Always `000000`	1st Source Register	2nd Source Register	Destination Register	Shift Amount (0 if unused)	ALU Function Code

PART 1: R-TYPE ASSEMBLY TO MACHINE LANGUAGE ENCODING

Question 1: Shift Left Logical Encoding

Mark 10.00 out of 315.00

[FILL IN THE BLANK - MACHINE ENCODING]

Technique / Concept Hint: MIPS shift instructions (`sll`): opcode=`000000`, rs=`00000` (unused), rt=source register, rd=destination register, shamt=immediate shift count, funct=`000000`.

Convert the MIPS assembly instruction `sll $t2, $t0, 4` to its 32-bit machine language representation:

Machine instruction: **0b** _____

Question 2: Shift Instruction Field Semantics

Mark 10.00 out of 315.00

[MULTIPLE CHOICE - SELECT ONE]

Technique / Concept Hint: MIPS R-type encoding conventions: identifying why the first source register field (`rs`) is set to all zeros during immediate shift operations (`sll`, `srl`, `sra`).

In the standard MIPS R-type instruction format, what is the value and role of the `rs` field (bits 25-21) in the instruction `sll $t0, $t1, 8`?

- ◯ a. It holds register `$t1` (9) as the primary ALU operand.
- ◯ b. It is set to `00000` because the shift amount is specified by the `shamt` field rather than a register.
- ◯ c. It holds the immediate shift value 8.
- ◯ d. It designates the memory segment base register.

Question 3: Bitwise AND Encoding

Mark 10.00 out of 315.00

[FILL IN THE BLANK - MACHINE ENCODING]

Technique / Concept Hint: Bitwise R-type encoding: opcode=`000000`, rs=1st source, rt=2nd source, rd=destination, shamt=`00000`, funct=`100100` (0x24).

Convert the MIPS assembly instruction `and $s0, $t1, $t2` to its 32-bit machine language representation:

Machine instruction: **0b** _____

Question 4: Logical Bitwise Hardware Properties

Mark 10.00 out of 315.00

[SELECT ALL THAT APPLY]

Technique / Concept Hint: MIPS bitwise operations: verifying that logical instructions (`and`, `or`, `nor`, `xor`) operate independently bit-by-bit, produce no arithmetic overflow exceptions, and use a shamt of zero.

Which of the following statements regarding MIPS R-type bitwise logical instructions are **TRUE**? (Select all that apply.)

- [ ] a. Bitwise logical instructions operate bit-by-bit independently across all 32 bit positions.
- [ ] b. Bitwise logical instructions can trigger a hardware integer overflow exception.
- [ ] c. In all standard non-shift R-type logical instructions, the `shamt` field is set to `00000`.
- [ ] d. The `nor` instruction can be used to implement bitwise NOT by setting one source register to `$zero`.

Question 5: Bitwise NOR Encoding

Mark 10.00 out of 315.00

[FILL IN THE BLANK - MACHINE ENCODING]

Technique / Concept Hint: NOR instruction encoding: opcode=`000000`, rs=`$t4` (12), rt=`$t5` (13), rd=`$t0` (8), shamt=`00000`, funct=`100111` (0x27).

Convert the MIPS assembly instruction `nor $t0, $t4, $t5` to its 32-bit machine language representation:

Machine instruction: **0b** _____

Question 6: Bitwise NOT Synthesis

Mark 10.00 out of 315.00

[MULTIPLE CHOICE - SELECT ONE]

Technique / Concept Hint: Synthesis of pseudoinstructions: evaluating how MIPS synthesizes `not $t0, $t1` using the elemental hardware primitive `nor`.

MIPS architectures do not provide a dedicated hardware `not` instruction. How is the pseudoinstruction `not $t0, $t1` synthesized using elemental R-type hardware?

- ◯ a. `nor $t0, $t1, $zero`
- ◯ b. `xor $t0, $t1, $zero`
- ◯ c. `and $t0, $t1, $zero`
- ◯ d. `subu $t0, $zero, $t1`

Question 7: Variable Shift Right Logical Encoding

Mark 10.00 out of 315.00

[FILL IN THE BLANK - MACHINE ENCODING]

Technique / Concept Hint: Variable shift encoding (`srlv rd, rt, rs`): rs specifies the register holding the shift count, rt specifies the value to be shifted, shamt=`00000`, funct=`000110` (0x06).

Convert the MIPS assembly instruction `srlv $s1, $t2, $t3` to its 32-bit machine language representation:

Machine instruction: **0b** _____

Question 8: Variable Shift Operand Mapping

Mark 10.00 out of 315.00

[MULTIPLE CHOICE - SELECT ONE]

Technique / Concept Hint: Identifying register positions in variable shifts: in `srlv rd, rt, rs`, the shift amount comes from register `rs`, while the data to be shifted comes from `rt`.

In the MIPS instruction `srlv $t7, $t9, $t8`, where `$t8` contains decimal 12:

- ◯ a. Register `$t9` is shifted right by 12 bit positions, and the result is stored in `$t7` .
- ◯ b. Register `$t8` is shifted right by the value in `$t9` , and stored in `$t7` .
- ◯ c. Register `$t7` is shifted left by 12 bit positions.
- ◯ d. The instruction uses the `shamt` field to encode the value 12 directly.

Question 9: Unsigned Add Encoding

Mark 10.00 out of 315.00

[FILL IN THE BLANK - MACHINE ENCODING]

Technique / Concept Hint: R-type addu encoding: opcode=`000000`, rs=1st source, rt=2nd source, rd=destination, shamt=`00000`, funct=`100001` (0x21).

Convert the MIPS assembly instruction `addu $t3, $t1, $t2` to its 32-bit machine language representation:

Machine instruction: **0b** _____

Question 10: Signed vs. Unsigned Instruction Behavior

Mark 10.00 out of 315.00

[SELECT ALL THAT APPLY]

Technique / Concept Hint: Exception handling mechanics: `add` and `sub` trap on two's complement overflow, whereas `addu` and `subu` never trap on overflow.

Which of the following statements regarding the distinction between MIPS `add` and `addu` instructions are **TRUE**? (Select all that apply.)

- ☐ a. `add` causes an arithmetic overflow exception (trap) when the signed result exceeds 32-bit boundaries.
- ☐ b. `addu` executes identical binary addition as `add` on the ALU hardware, but never causes an overflow exception.
- ☐ c. `addu` forces both register operands to be positive before adding them.
- ☐ d. C and Java compilers targeting MIPS typically emit `addu` and `subu` because language standards do not require hardware overflow traps.

PART 2: 4-BIT & 32-BIT ARITHMETIC EXECUTION & FLAG EVALUATION

Question 11: 4-Bit Architecture Addition (-6 + 2)

Mark 10.00 out of 315.00

[FILL IN THE BLANK - EXECUTION TRACE]

Technique / Concept Hint: 4-bit two's complement execution: convert negative operand to 4-bit binary (`1010`), add positive operand, check for carry-out, borrow, overflow, and underflow.

Let's execute the instruction `add $t5, $t1, $t2`. Given `$t1 = −6` and `$t2 = 2` on a **4-bit architecture**:

`$t1 = 0b` [ ] | `$t2 = 0b` [ ] | `$t5 = 0b` [ ]

Carry flag = [ ] Borrow flag = [ ] Overflow flag = [ ]

Underflow flag = [ ]

Question 12: 4-Bit Representation Limits

Mark 10.00 out of 315.00

[MULTIPLE CHOICE - SELECT ONE]

Technique / Concept Hint: Numerical limits of 4-bit signed representations: range [-8, +7].

In a 4-bit architecture, what would occur if the instruction executed was `add $t5, $t1, $t2` with `$t1 = −6` and `$t2 = −3`?

- ◯ a. The result -9 produces a negative overflow because -9 is below the lower bound of -8.
- ◯ b. The result executes without flags because -9 fits into 4 bits.
- ◯ c. The processor automatically converts both numbers to unsigned 16-bit values.
- ◯ d. The Borrow flag is set to 1 and the Zero flag is set to 1.

Question 13: 32-Bit Signed Addition (-12 + 3)

Mark 10.00 out of 315.00

[FILL IN THE BLANK - EXECUTION TRACE]

Technique / Concept Hint: 32-bit hex sign-extension: negative integers have leading `0xFFFFFF..`; compute 32-bit sum in hex and evaluate status flags.

Let's execute the instruction `add $t7, $t3, $t4`. Given `$t3 = −12` and `$t4 = 3` on a **32-bit architecture**:

`$t3 = 0x` [ ] | `$t4 = 0x` [ ] | `$t7 = 0x`

[ ]

Carry flag = [ ] Borrow flag = [ ] Overflow flag = [ ]

Underflow flag = [ ]

Question 14: 32-Bit Hexadecimal Sign Patterns

Mark 10.00 out of 315.00

[SELECT ALL THAT APPLY]

Technique / Concept Hint: Sign extension in two's complement 32-bit words: recognizing negative numbers by leading hex digits 8, 9, A, B, C, D, E, F (MSB = 1).

Which of the following 32-bit hexadecimal values represent negative integers in standard two's complement notation? (Select all that apply.)

- [ ] a. `0xFFFFFFF8` (-8)
- [ ] b. `0x80000000` (-2^{31})
- [ ] c. `0x7FFFFFFF` ($+2^{31} - 1$)
- [ ] d. `0x00000001` (+1)

Question 15: 32-Bit addu Execution & Machine Encoding (-5 + 4)

Mark 25.00 out of 315.00

[FILL IN THE BLANK - TRACE & ENCODING]

Technique / Concept Hint: Full machine state tracing: compute 32-bit hexadecimal, decimal result, flags, and 32-bit instruction encoding hex.

Let's execute the instruction `addu $t8, $t1, $t2`. Given `$t1 = −5` and `$t2 = 4` on a **32-bit architecture**:

`$t1 = 0x` [ ] | `$t2 = 0x` [ ] | `$t8 = 0x` [ ]

Result in decimal: `$t8 = 0d` [ ]

Carry flag = [ ] Borrow flag = [ ] Overflow flag = [ ]

Underflow flag = [ ]

Machine instruction encoding for `addu $t8, $t1, $t2` : **0x** [ ]

Question 16: Overflow Handling in addu

Mark 10.00 out of 315.00

[MULTIPLE CHOICE - SELECT ONE]

Technique / Concept Hint: Microarchitectural definition of addu: arithmetic is performed identically, but hardware trap signaling is disabled.

What does the MIPS ALU do when the mathematical sum of an addu instruction exceeds the representable signed range?

- ( ) a. It triggers an operating system kernel trap to handle the exception.
- ( ) b. It silently stores the truncated 32-bit result in the destination register without raising an exception.
- ( ) c. It clears the destination register to all zeros.
- ( ) d. It freezes the pipeline until a manual hardware reset occurs.

Question 17: Carry Generation with Negative Addition (-16 + -1)

Mark 25.00 out of 315.00

[FILL IN THE BLANK - TRACE & ENCODING]

Technique / Concept Hint: Two's complement addition of negative numbers: adding two numbers with MSB=1 generates a carry-out of the 32nd bit, setting Carry = 1.

Let's execute the instruction `addu $t6, $t4, $t5`. Given `$t4 = −16` and `$t5 = −1` on a **32-bit architecture**:

`$t4 = 0x` _____ | `$t5 = 0x` _____ | `$t6 = 0x` _____

Carry flag = _____ Borrow flag = _____ Overflow flag = _____

Underflow flag = _____

Machine instruction encoding for `addu $t6, $t4, $t5` : **0x** _____

Question 18: Carry vs. Overflow During Negative Sums

Mark 10.00 out of 315.00

[SELECT ALL THAT APPLY]

Technique / Concept Hint: Adder carry behavior on negative operands: in two's complement, adding any two negative numbers always produces an unsigned carry-out of 1, even if signed overflow is 0.

Which of the following statements regarding the addition of two negative numbers (e.g., -8 and -1) in 32-bit two's complement are **TRUE**? (Select all that apply.)

- [ ] a. A Carry-out of 1 is generated by the 32-bit adder because both operands have values ≥ 2^{31} when interpreted as unsigned integers.
- [ ] b. Signed overflow occurs because the result remains negative.
- [ ] c. No signed overflow occurs as long as the mathematical sum is greater than or equal to -2^{31}.
- [ ] d. The Borrow flag is set to 1 because addition is mathematically the opposite of subtraction.

Question 19: Positive Addition Trace (14 + 7)

Mark 10.00 out of 315.00

[FILL IN THE BLANK - EXECUTION TRACE]

Technique / Concept Hint: Small positive integer addition: verify zero-extended 32-bit hex values and ensure all condition flags evaluate to 0.

Let's execute the instruction `add $t2, $t0, $t1`. Given `$t0 = 14` and `$t1 = 7` on a **32-bit architecture**:

`$t0 = 0x` _____ | `$t1 = 0x` _____ | `$t2 = 0x` _____

Result in decimal: `$t2 = 0d` _____

Question 20: ALU Zero Flag Detection

Mark 10.00 out of 315.00

[MULTIPLE CHOICE - SELECT ONE]

Technique / Concept Hint: ALU status generation: Zero flag hardware implementation using a 32-input NOR tree across all output bits.

In the MIPS processor datapath, how does the ALU hardware determine whether the **Zero flag** should be asserted?

- a. By feeding all 32 bits of the ALU result into a multi-input NOR gate tree (outputs 1 if and only if all bits are 0).
- b. By comparing the Program Counter against address zero.
- c. By checking whether register `$zero` was referenced as a destination.
- d. By checking whether the carry-out from bit 15 is 0.

PART 3: SUBTRACTION, BITWISE MASKING & LOGICAL SHIFTS

Question 21: Unsigned Subtraction Trace (8 - 20)

Mark 25.00 out of 315.00

[FILL IN THE BLANK - TRACE & ENCODING]

Technique / Concept Hint: Negative subtraction in `subu`: 8 - 20 = -12; represent in 32-bit hex and binary; determine condition evidence; encode `subu` (funct=0x23).

Let's execute the instruction `subu $t0, $t1, $t2` with values `$t1 = 8` and `$t2 = 20` on a 32-bit MIPS architecture:

`$t1 = 0x` [ ] | `$t2 = 0x` [ ] | `$t0 = 0x` [ ]

Carry (Yes / No): [ ] Overflow (Yes / No): [ ] Underflow (Yes / No): [ ]

Zero (Yes / No): [ ]

Machine instruction encoding for `subu $t0, $t1, $t2` : **0x** [ ]

Question 22: Hardware Subtraction Implementation

Mark 10.00 out of 315.00

[SELECT ALL THAT APPLY]

Technique / Concept Hint: ALU subtraction execution: A - B implemented via A + ~B + 1; lack of signed overflow traps in `subu`.

Which of the following statements regarding the execution of `subu` in MIPS are **TRUE**? (Select all that apply.)

- [ ] a. The ALU calculates A - B by inverting operand B and setting the carry-in bit to 1 (A + ~B + 1).
- [ ] b. If 8 - 20 is executed, the result wraps around to `0xFFFFFFF4` without throwing an overflow exception.
- [ ] c. If `sub` were used instead of `subu`, subtracting a positive number from a positive number would trigger an overflow exception.
- [ ] d. `subu` stands for "Subtract Unsigned" and ignores signed overflow.

Question 23: Bitwise NOR Masking

Mark 15.00 out of 315.00

[FILL IN THE BLANK - BITWISE EXECUTION]

Technique / Concept Hint: Bitwise NOR evaluation: NOR(A, B) = ~(A | B); compute OR of hexadecimal patterns and invert every bit.

In a MIPS architecture, execute the instruction `nor $t0, $t1, $t2`. Given:

```
$t1 = 0x00FF00FF
$t2 = 0x0F0F0F0F
```

Demonstrate the resulting value in register `$t0`:

`$t0 in Hex = 0x` [ ]

Highest 8 bits of `$t0` in binary: `0b` [ ]

Question 24: De Morgan's Laws in Digital Logic

Mark 10.00 out of 315.00

[MULTIPLE CHOICE - SELECT ONE]

Technique / Concept Hint: Boolean equivalence: by De Morgan's laws, ~(A | B) = ~A & ~B.

According to De Morgan's Laws, which Boolean logical expression is mathematically equivalent to the MIPS instruction `nor $t0, $t1, $t2`?

- ◯ a. (~$t1) & (~$t2) (Bitwise NOT $t1 AND Bitwise NOT $t2)
- ◯ b. (~$t1) | (~$t2) (Bitwise NOT $t1 OR Bitwise NOT $t2)
- ◯ c. $t1 & $t2
- ◯ d. ~($t1 & $t2) (NAND)

Question 25: Shift Left Logical Trace

Mark 15.00 out of 315.00

[FILL IN THE BLANK - ARITHMETIC TRACE]

Technique / Concept Hint: Logical left shift: shifting a binary integer left by k bits multiplies its numerical value by 2^k ($X \times 2^k$).

In the MIPS architecture, execute the instruction `sll $t0, $t1, 6` where `$t1 = 16`:

Initial `$t1` in decimal = 16 → Initial `$t1` in binary = `0b` [ ]

Resulting `$t0` in binary = `0b` [ ]

Result in decimal format: `$t0 =` [ ]

Question 26: Logical vs. Arithmetic Shifts

Mark 10.00 out of 315.00

[MULTIPLE CHOICE - SELECT ONE]

Technique / Concept Hint: Shift behavior: Logical shifts (`srl`) insert zeros on the left, while Arithmetic shifts (`sra`) replicate the sign bit (MSB).

What is the fundamental difference between the MIPS instructions `srl $t0, $t1, 2` (Shift Right Logical) and `sra $t0, $t1, 2` (Shift Right Arithmetic)?

a. `srl` shifts in 0 bits from the left, while `sra` replicates the sign bit (bit 31) into vacated positions to preserve two's complement sign.

b. `srl` works with registers, while `sra` works strictly with immediate values.

c. `sra` shifts left instead of right.

d. `srl` throws an overflow exception on negative values.

Instructor Answer Key & Comprehensive Solutions

Maximum Marks: 315.00

Quiz #7 (Unit 7 - Extended) - Step-by-Step Encoding & Execution Guide

---

Question 1 (10.00 Marks) — sll $t2, $t0, 4 Encoding

Correct Answer: 00000000000010000101000100000000 (or 0x00085100)

Bitfield derivation:

- Opcode: `000000` (6 bits)
- rs: `00000` (unused for sll, 5 bits)
- rt: `$t0` = 8 = `01000` (5 bits)
- rd: `$t2` = 10 = `01010` (5 bits)
- shamt: 4 = `00100` (5 bits)
- funct: `000000` (6 bits)

Combined binary: `000000 00000 01000 01010 00100 000000`.

Question 2 (10.00 Marks) — Shift rs Field Semantics

Correct Answer: b

Explanation: Standard MIPS immediate shift instructions (`sll`, `srl`, `sra`) do not use an `rs` source register because the shift amount is supplied directly by the 5-bit `shamt` field; hardware mandates `rs = 00000`.

Question 3 (10.00 Marks) — and $s0, $t1, $t2 Encoding

Correct Answer: 00000001001010101000000000100100 (or 0x012a8024)

Bitfield derivation:

- Opcode: `000000`
- rs: `$t1` = 9 = `01001`
- rt: `$t2` = 10 = `01010`
- rd: `$s0` = 16 = `10000`
- shamt: `00000`
- funct: `100100` (0x24 for AND)

Combined binary: `000000 01001 01010 10000 00000 100100`.

Question 4 (10.00 Marks) — Logical Bitwise Properties

Correct Answers: a, c, d

Explanation: Logical instructions operate independently across bits without carry propagation, so they cannot trigger arithmetic overflow exceptions (**b is FALSE**). `shamt` is `00000` for non-shifts, and `nor $d, $s, $zero` computes ~(s | 0) = ~s (NOT).

Question 5 (10.00 Marks) — nor $t0, $t4, $t5 Encoding

Correct Answer: 00000001100011010100000000100111 (or 0x018d4027)

Bitfield derivation:
• Opcode: `000000`
• rs: `$t4` = 12 = `01100`
• rt: `$t5` = 13 = `01101`
• rd: `$t0` = 8 = `01000`
• shamt: `00000`
• funct: `100111` (0x27 for NOR)
Combined binary: `000000 01100 01101 01000 00000 100111`.

Question 6 (10.00 Marks) — NOT Synthesis

Correct Answer: a. nor $t0, $t1, $zero

Explanation: By Boolean identity, NOR(X, 0) = ~(X | 0) = ~X. MIPS assemblers synthesize the `not $t0, $t1` macro using `nor $t0, $t1, $zero`.

Question 7 (10.00 Marks) — srlv $s1, $t2, $t3 Encoding

Correct Answer: 00000001011010101000100000000110 (or 0x016a8806)

Bitfield derivation for variable shift (srlv rd, rt, rs):
• Opcode: `000000`
• rs: `$t3` = 11 = `01011` (holding shift count)
• rt: `$t2` = 10 = `01010` (register to be shifted)
• rd: `$s1` = 17 = `10001` (destination register)
• shamt: `00000`
• funct: `000110` (0x06 for SRLV)
Combined binary: `000000 01011 01010 10001 00000 000110`.

Question 8 (10.00 Marks) — srlv Operand Roles

Correct Answer: a

Explanation: In the MIPS architecture, `srlv rd, rt, rs` shifts the contents of register `rt` right by the number of bits specified in the 5 lowest bits of register `rs`, placing the result in `rd`.

Question 9 (10.00 Marks) — addu $t3, $t1, $t2 Encoding

Correct Answer: 00000001001010100101100000100001 (or 0x012a5821)

Bitfield derivation:
• Opcode: `000000`
• rs: `$t1` = 9 = `01001`
• rt: `$t2` = 10 = `01010`
• rd: `$t3` = 11 = `01011`
• shamt: `00000`
• funct: `100001` (0x21 for ADDU)
Combined binary: `000000 01001 01010 01011 00000 100001`.

Question 10 (10.00 Marks) — add vs. addu Behavior

Correct Answers: a, b, d

Explanation: Both instructions perform identical two's complement binary addition in hardware. The only distinction is that add traps on signed overflow while addu does not. Operand signs are never modified (**c is FALSE**).

Question 11 (10.00 Marks) — 4-Bit Execution (-6 + 2)

$t1 = [1010] | $t2 = [0010] | $t5 = [1100] (Decimal -4)
Carry = [0] | Borrow = [0] | Overflow = [0] | Underflow = [0]

Explanation: -6 = 1010_2, +2 = 0010_2. Sum: $1010_2 + 0010_2 = 1100_2$ = -4. No carry-out or overflow occurs.

Question 12 (10.00 Marks) — 4-Bit Range Limits

Correct Answer: a

Explanation: The valid range for 4-bit two's complement is [-8, +7]. The sum -6 + (-3) = -9 < -8, resulting in a negative signed overflow (wrapping to +7).

Question 13 (10.00 Marks) — 32-Bit Execution (-12 + 3)

$t3 = [fffffff4] | $t4 = [00000003] | $t7 = [fffffff7]
Carry = [0] | Borrow = [0] | Overflow = [0] | Underflow = [0]

Explanation: -12 = 0xFFFFFFF4, +3 = 0x00000003. Sum = 0xFFFFFFF7 (-9). Adding operands of opposite signs produces no carry or overflow.

Question 14 (10.00 Marks) — 32-Bit Hex Sign Recognition

Correct Answers: a, b

Explanation: In two's complement, numbers with MSB = 1 are negative. In hexadecimal, leading digits 8 through F indicate that bit 31 is 1. `0x7FFFFFFF` begins with 0111, so it is positive.

Question 15 (25.00 Marks) — 32-Bit addu Trace (-5 + 4)

$t1 = [fffffffb] | $t2 = [00000004] | $t8 = [ffffffff] | Decimal: [-1]
Carry = [0] | Borrow = [0] | Overflow = [0] | Underflow = [0]
Encoding: [0x012ac021] (or 012ac021)

Explanation: -5 = 0xFFFFFFFB, +4 = 0x00000004. Sum = 0xFFFFFFFF (-1). Encoding for `addu $t8, $t1, $t2`: rs=$t1 (9), rt=$t2 (10), rd=$t8 (24), funct=0x21 → `0x012ac021`.

Question 16 (10.00 Marks) — addu Overflow Behavior

Correct Answer: b

Explanation: addu does not signal overflow exceptions; the ALU performs modulo 2^{32} arithmetic and deposits the lowest 32 bits into the destination register.

Question 17 (25.00 Marks) — addu Negative Carry (-16 + -1)

$t4 = [fffffff0] | $t5 = [ffffffff] | $t6 = [ffffffef]
Carry = [1] | Borrow = [0] | Overflow = [0] | Underflow = [0]

Encoding: [0x018df021] (or 018df021)

Explanation: -16 = 0xFFFFFFF0, -1 = 0xFFFFFFFF. 0xFFFFFFF0 + 0xFFFFFFFF = 0x1FFFFFFEF. The 33rd bit produces Carry = 1. Encoding for `addu $t6, $t4, $t5`: rs=$t4 (12), rt=$t5 (13), rd=$t6 (14), funct=0x21 → `0x018df021`.

Question 18 (10.00 Marks) — Carry on Negative Addition

Correct Answers: a, c

Explanation: Two negative numbers have bit 31 set (1 + 1 = 10_2), guaranteeing a Carry-out of 1 from the 32-bit adder. No signed overflow occurs unless the sum drops below -2^{31}.

Question 19 (10.00 Marks) — Positive Addition Trace (14 + 7)

$t0 = [0000000e] | $t1 = [00000007] | $t2 = [00000015] | Decimal: [21]

Explanation: 14 = 0x0000000E, 7 = 0x00000007. Sum: 14 + 7 = 21 = 0x00000015.

Question 20 (10.00 Marks) — ALU Zero Flag Circuit

Correct Answer: a

Explanation: The Zero flag is implemented by taking the logical NOR of all 32 output bits of the ALU; if any bit is 1, NOR produces 0, and if all bits are 0, NOR asserts 1.

Question 21 (25.00 Marks) — subu Trace (8 - 20)

$t1 = [00000008] | $t2 = [00000014] | $t0 = [fffffff4]
Carry: [No] | Overflow: [No] | Underflow: [No] | Zero: [No]
Encoding: [0x012a4023] (or 012a4023)

Explanation: 8 - 20 = -12 = 0xFFFFFFF4. Evidence: No carry, no overflow, no underflow, no zero. Encoding for `subu $t0, $t1, $t2`: rs=$t1 (9), rt=$t2 (10), rd=$t0 (8), funct=0x23 → `0x012a4023`.

Question 22 (10.00 Marks) — subu Hardware Implementation

Correct Answers: a, b, d

Explanation: Subtraction uses A + ~B + 1. `subu` suppresses overflow traps. Subtracting two positive numbers cannot cause signed overflow in two's complement (**c is FALSE**).

Question 23 (15.00 Marks) — nor $t0, $t1, $t2 Masking

$t0 in Hex = [f000f000] | Highest 8 bits binary: [11110000]

Step-by-step bitwise logic:
$t1 | $t2 = 0x00FF00FF | 0x0F0F0F0F = 0x0FFF0FFF.
Bitwise inversion: ~(0x0FFF0FFF) = 0xF000F000.
Highest 8 bits: 0xF0 = 11110000_2.

Question 24 (10.00 Marks) — De Morgan's Law

Correct Answer: a. (~$t1) & (~$t2)

Explanation: De Morgan's theorem states that ~(A | B) = ~A & ~B.

Question 25 (15.00 Marks) — sll $t0, $t1, 6 Trace

Initial $t1 = [00000000000000000000000000010000]
Result $t0 binary = [00000000000000000000010000000000]
Decimal: [1024]

Explanation: 16 = 2^4. Shifting left by 6 bit positions multiplies by 2^6 = 64. 16 × 64 = 1024 = 2^{10}.

Question 26 (10.00 Marks) — srl vs. sra

Correct Answer: a

Explanation: `srl` (logical) always fills vacated high-order bit positions with 0, while `sra` (arithmetic) sign-extends by replicating bit 31, preserving the negative sign in signed two's complement divisions.
